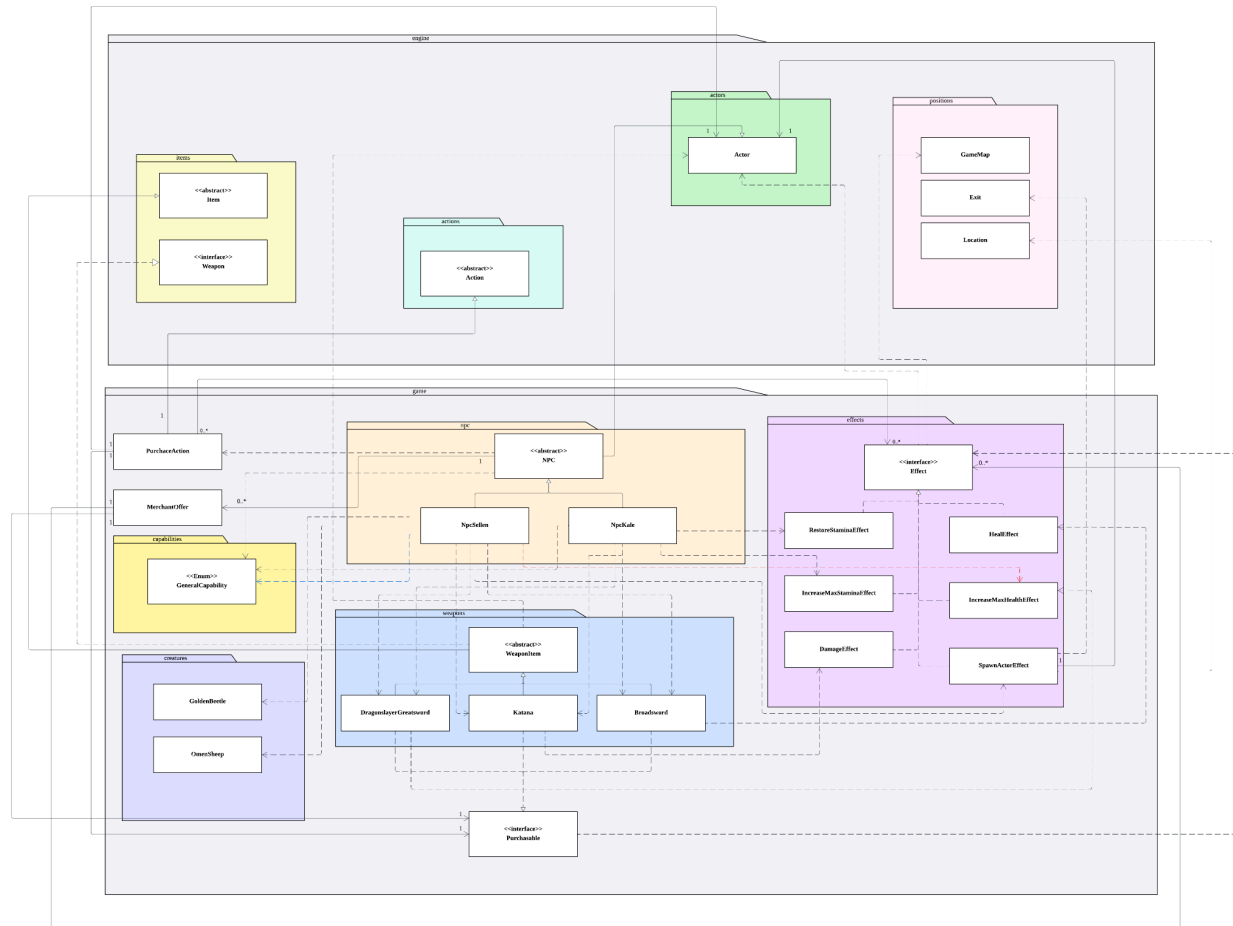


Design Rationale: Purchasing System (REQ4)



1. Introduction

This rationale details the design for a system enabling Player weapon purchases from NPC merchants (REQ4). It covers the purchase process, layered base and merchant-specific effects, UI constraints (single-menu, one-tick actions), and future extensibility, prioritizing robust, maintainable, object-oriented design.

2. Design Goals

- **Single-Menu Purchase:** Ensure all buying actions are available in the same menu when interacting with a merchant, with each purchase costing one standard game tick.
- **Layered & Complex Effects:** Implement a system to apply base effects inherent to a weapon upon purchase (like Broadsword always heals 10 HP),

plus additional, distinct effects dependent on the specific merchant selling the item (like Sellen's Broadsword offer also grants +20 Max HP).

- **Extensibility & Maintainability (OCP):** Design the system so that new merchants, new purchasable items (weapons or other types), and new purchase-related effects can be added with minimal to no changes to existing, stable system components.

3. Design Analysis & Decisions

1. Core Purchase & Effect System

This feature addresses the overall architecture for managing item purchases, including varying costs and the application of multiple, layered effects.

Option A: Monolithic Logic in Player and PurchaseAction

- **Description:** All logic for identifying items, checking prices, deducting runes, transferring items, and applying all specific base and merchant-dependent effects would be hardcoded within a single large class, likely the Player class and PurchaseAction.
- **Maintainability:** Extremely poor. This class would become a super complex class, difficult to understand, modify, and test.

Pros	Cons
Simpler for a small, static set of items.	Violates SRP (complex Player logic) and OCP (modifications needed for new items).

Option B: Interface-Driven Design

- **Description:** Purchasable interface defines weapon base effects with methods like getting the base effects and transferring the item to buyer. MerchantOffer holds a Purchasable item, price, and merchant-specific effects in an ArrayList. PurchaseAction will handle runes, transfer, and effect application.
- **Maintainability:** High. New items or effects are added without modifying core classes.

Pros	Cons
Adheres to SRP, OCP, and DRY, isolating components for testability.	Extensible for new merchants and items.

Decision & Justification

- **Chosen Design:** Option B.
- **Rationale:** Option B ensures modularity and extensibility (SRP, OCP). For instance, a new Fire Sword with a Burn effect can implement Purchasable and use a new MerchantOffer without changing PurchaseAction (OCP). If Kale changes his Broadsword offer to include a StrengthBuffEffect, only Kale's MerchantOffer instance for the Broadsword needs to be updated with this new Effect. The Broadsword class and the PurchaseAction remain untouched.

3.2. UI Integration & Action Cost

This feature ensures purchases are integrated into the main action menu and cost one game tick.

Option A: Npc Base Class Action Integration

- **Description:** The Npc base class's allowableActions() generates PurchaseAction instances for each MerchantOffer, adding them to the main menu. The engine's action lifecycle ensures one-tick cost. These actions will be processed if the NPC has CAN_SELL capability, and the item and price will be added to the list returned by the NPC's allowableActions method if the Player has CAN_BUY capability
- **Maintainability:** High. Leverages the engine's action system for simplicity.

Pros	Cons
Uses existing engine Action system meeting requirements. NPCs define actions, PurchaseAction executes. (SRP)	Long action menu lists for many items.

Option B: Centralized Purchase Manager

- **Description:** A single PurchaseManager checks for nearby merchants, hardcodes their offers, and generates PurchaseAction instances for the main menu in the game loop.
- **Maintainability:** Poor. PurchaseManager requires modifications for new merchants or items, complicating updates.

Pros	Cons
Centralizes buying logic for a small, static set of merchants.	Violates SRP (mixes action generation and buying logic) and OCP (new offers modify PurchaseManager).
Consistent UX, making the game feel more cohesive.	Duplicates logic (DRY violation) and tightly couples to merchant offers.

Decision & Justification

- **Chosen Design:** Option A.
- **Rationale:** Option B fulfills requirements by integrating buying into the main menu via `Npc.allowableActions()`, maintaining UX consistency and more robust integration with the existing engine action system. For instance, a new `TradeAction` can extend `allowableActions()` without altering the menu system (OCP). It treats purchasing as a clear action directly offered by the merchant for each item, fitting naturally into the game's established model and ensuring the "no separate menu" rule is met.

3.3. Effect Implementation Details

This feature focuses on the implementation of specific purchase effects like healing, stat changes, and actor spawning, utilizing the Effect interface and its concrete classes.

Option A: Effect Logic in PurchaseAction

- **Description:** The PurchaseAction.execute() method would contain large if/else if blocks or a switch statement to determine which item is being bought and then directly apply all associated base and merchant-specific effects by calling Player/map methods.
- **Maintainability:** Poor. Complex logic requires modifications for new effects.

Pros	Cons
Simpler for few, static effects.	Violates SRP & DRY. Becomes unmaintainable and error-prone as effects grow in number or complexity.

Option B: Effect Interface and Concrete Classes

- **Description:** Effect interface defines applyEffect(). Concrete classes (like HealEffect, SpawnActorEffect) handle specific effects. Purchasable items (in getBasePurchaseEffects()) and MerchantOffer objects (in their list of additional effects) hold lists of these Effect objects. The PurchaseAction iterates through these lists and apply the effect for each one, polymorphically executing the correct outcome.
- **Maintainability:** High. New effects are added without modifying existing code.

Pros	Cons
OCP & SRP compliant. Promotes DRY. Flexible, compositional, and highly	Results in a larger number of small, focused classes.

maintainable. Testing individual effects is straightforward.	
--	--

Decision & Justification

- **Chosen Design:** Option B.
- **Rationale:** Option B, utilizing the Effect interface and concrete classes, is chosen for its flexibility, maintainability, and adherence to SOLID principles (OCP, SRP). For instance, a new GrantsInvisibilityEffect can be added without altering PurchaseAction (OCP). If a new effect is needed, only a new Effect class is required, ensuring maintainability over Option A's rigid conditionals.

4. Conclusion

This design for the purchasing system, centered around the Purchasable interface for items, MerchantOffer to define specific deals, a cohesive PurchaseAction for transaction orchestration, and a flexible Effect system (Strategy Pattern) for outcome management, prioritizing SRP, OCP, and DRY. This modular system supports future items and effects, aligning with REQ4's UML and requirements.